Personal Learning Journey Report

IT3091 Machine Learning — Individual Assessment Deliverable

Student ID: IT24102664

Group ID: 2026-AI-36

Project Title: ChefCast AI — Restaurant Revenue & Product Demand Forecasting

Assigned Notebooks: NB02 (Lens 1 — Revenue Data Preparation & Feature Engineering) and NB08 (Lens 2 — Product Demand Validation, Stacking & Model Selection)

Submission Date: October 2026

Section 1: My Technical Role & Specific Contributions

NB02 — Lens 1: Revenue Data Preparation & Feature Engineering

My responsibility in NB02 was to transform the cleaned POS transaction data into a machine-learning-ready feature matrix for Lens 1. This notebook is the engineering core of the entire pipeline — every model trained in NB03 and NB04 depends on the quality and integrity of the feature matrix I built here.

Specific work I performed:

1. Temporal Aggregation & Calendar Reindexing:

I built the pipeline that converts the 125,912 cleaned transaction records into a 895-day continuous daily revenue time series. This involved grouping transactions by date and summing `LineRevenue` across the 11 target items. Critically, after aggregation, I reindexed the series against a complete calendar spanning 2024-03-27 to 2026-09-07, inserting `0.0` revenue on dates with no qualifying sales. This zero-fill decision was non-trivial — I validated it by comparing two pipeline variants (with and without zero-fill) and confirming that zero-filling reduces the 7-day WAPE on a preliminary Ridge model by approximately 2.3% due to preserved temporal continuity.

2. The Critical Leakage Prevention Architecture:

The most important technical contribution of NB02 was implementing a formally leak-free feature engineering pipeline. My approach:

```
# Shift series by 1 before computing ALL rolling statistics revenue_shifted = daily_revenue.shift(1) # Rolling window features computed only on past observations roll_mean_7 = revenue_shifted.rolling(7).mean() roll_std_7 = revenue_shifted.rolling(7).std() roll_mean_14 = revenue_shifted.rolling(14).mean() roll_mean_30 = revenue_shifted.rolling(30).mean()
```

This single design decision prevents a class of errors that would make the model appear far more accurate in testing than it actually is in deployment. I validated leakage prevention by confirming that at prediction time ***t***, the value of `roll_mean_7[t]` equals the average of ***y_{t-1}*** through ***y_{t-7}*** exactly — never including ***y_t*** itself.

3. Cyclical Trigonometric Encoding:

I implemented the sine/cosine transformation for day-of-week and month:

$***{sin_dow}(t) = \sin\!\left(\frac{2\pi · {dow}(t)}{7}\right), {cos_dow}(t) = \cos\!\left(\frac{2\pi · {dow}(t)}{7}\right)***$

I chose this encoding over simple one-hot encoding for two reasons: (i) it preserves the circular continuity between Sunday (dow=6) and Monday (dow=0), and (ii) it reduces feature dimensionality from 7 binary columns to 2 continuous columns, which benefits Ridge Regression's L2 penalty calculation by not artificially inflating coefficient counts.

4. Autoregressive Lag Features:

I engineered lag features at ***t-1, t-2, t-3, t-7, t-14, t-21***. The lag selection rationale:

- `lag_1, lag_2, lag_3`: Capture immediate short-term momentum.
- `lag_7`: Captures the same day-of-week pattern from the previous week — the strongest single predictor as confirmed by LightGBM feature importance (6.55%).
- `lag_14, lag_21`: Capture fortnightly and tri-weekly patterns from payday cycles.

5. Momentum Ratio Features:

I devised two derived features to capture relative velocity:

$***{ratio_lag1_to_roll7} = \frac{y_{t-1}}{{roll_mean_7}(t) + 10^{-5}}***$

The ratio encodes whether recent demand is trending above or below the 7-day average — a meaningful operational signal that is dimensionless and scale-invariant across different dish price levels.

6. Final Feature Matrix Validation:

I performed a post-engineering leakage audit: for each feature column, I verified that the correlation between the feature value at ***t*** and ***y_{t+1}*** (future revenue) is within expected autoregressive bounds — and not higher than the correlation between the same feature and ***y_{t-1}*** (past revenue). Any feature showing anomalously high future correlation would indicate leakage.

Final output: Lens 1 feature matrix — 874 rows (post-21-day burn-in) × 30 features.

NB08 — Lens 2: Product Demand Validation, Stacking & Model Selection

My responsibility in NB08 was the most methodologically complex task in the entire project: implementing the time-series-safe stacking ensemble and conducting the formal model selection process for Lens 2.

Specific work I performed:

1. Chronological Validation Split Design:

I established the three-way chronological split for Lens 2: Training (2024-03-27 to 2025-12-18), Validation (2025-12-19 to 2026-04-28), Test (2026-04-29 to 2026-09-07). Crucially, I enforced that the split operates on **whole calendar dates** — meaning all 11 product rows belonging to a single date are always in the same split. This is critical: if Wednesday 2025-12-20 is in the validation set, all 11 dish rows for that date must be in the validation set together. Partial date splits would create information leakage between product panel rows.

2. Custom TimeSeriesStackingRegressor Implementation:

The standard `sklearn.StackingRegressor` uses randomized cross-validation internally, which creates temporal leakage on sequential data. I implemented a custom stacking class:

```
class TimeSeriesStackingRegressor: def fit(self, X_train, y_train): tscv = TimeSeriesSplit(n_splits=3) oof_predictions = np.zeros((len(X_train), len(self.base_models))) for fold, (train_idx, val_idx) in enumerate(tscv.split(X_train)): for j, model in enumerate(self.base_models): model_clone = clone(model) model_clone.fit(X_train[train_idx], y_train[train_idx]) oof_predictions[val_idx, j] = model_clone.predict(X_train[val_idx]) # Meta-learner sees only OOF predictions — no future contamination self.meta_learner.fit(oof_predictions, y_train) # Refit base models on full training data for inference for model in self.base_models: model.fit(X_train, y_train)
```

The `TimeSeriesSplit(n_splits=3)` ensures that within each internal cross-validation fold, training data strictly precedes validation data in time — preventing any future observation from appearing in the meta-learner's training inputs.

3. Champion Model Selection via Validation WAPE:

I evaluated all five models on the Validation set and ranked them strictly by Validation WAPE:

Rank	Model	Validation WAPE
1	**Ridge Regression**	**47.31%** ← Selected Champion
2	Stacking Ensemble	48.06%
3	Random Forest	48.26%
4	XGBoost	50.92%
5	LightGBM	54.56%

I documented that the test set was not consulted at this stage. The champion was committed to Ridge Regression based solely on Validation WAPE. Only then was the test set evaluated — revealing Ridge's unbiased Test WAPE of 44.38%.

4. Champion Retraining for Deployment:

After selection, I retrained the Ridge champion on the combined Train + Validation dataset (extended window: 2024-03-27 to 2026-04-28) before serialising it as `lens2_champion_model.joblib`. This maximises the amount of historical signal available to the production model without contaminating the test set evaluation.

Section 2: Critical Challenges I Overcame

Challenge 1: Implementing Truly Leak-Free Rolling Features

The conceptually simple statement "all features must use past data only" is surprisingly difficult to implement correctly in pandas. My first implementation computed `rolling(7).mean()` directly on the revenue series without shifting — which I caught during a leakage audit by noticing that the `roll_mean_7` value for row t included y_t when the window was not shifted. A single `.shift(1)` call before the `.rolling()` call corrected this, but understanding *why* the shift is necessary required careful thinking about pandas rolling window semantics and what "at index t" means in the computation.

Challenge 2: The sklearn StackingRegressor TimeSeriesSplit Incompatibility

When I first attempted to use `sklearn.StackingRegressor` with `TimeSeriesSplit` as the `cv` parameter, it raised a `ValueError: cross_val_predict only works for partitions of the data` — because `TimeSeriesSplit` creates overlapping indices that sklearn's internal validation cannot handle. Rather than falling back to random CV (which would introduce leakage), I researched the issue and implemented the custom `TimeSeriesStackingRegressor` from scratch. This forced me to truly understand the stacking mechanics from first principles rather than treating it as a black-box API call.

Challenge 3: Interpreting the Ridge vs. Stacking Result

On Lens 2, the Stacking Ensemble achieved a slightly better Test WAPE (44.35%) than Ridge (44.38%) — but Ridge was correctly selected as champion because it achieved a lower *Validation* WAPE (47.31% vs 48.06%). I had to clearly document why this apparent "suboptimal" selection is actually the *correct* methodological practice: selecting based on test performance would constitute test-set peeking, invalidating the unbiased evaluation. This distinction between validation-based selection and test-set evaluation is one of the most important methodological lessons of the project.

Section 3: Individual Learning Reflection (Fink's Taxonomy)

Foundational Knowledge

I entered this project with a textbook understanding of Ridge Regression as "linear regression with L2 regularization to prevent overfitting." I leave it with a deep operational understanding of *when* Ridge outperforms gradient boosting: in high-noise, low-signal-to-noise settings such as disaggregated food demand (5–20 portions/day), the controlled bias of regularized linear models produces better generalization than the expressive but overfit-prone leaf-wise splitting of LightGBM. This is the bias-variance tradeoff applied in a real, consequential context.

I also deeply understand temporal data leakage now — not as an abstract academic concern, but as a practical pipeline implementation challenge with specific code-level solutions (the `.shift(1)` pattern, `TimeSeriesSplit` for internal cross-validation).

Application

My biggest application learning was translating the abstract concept of "chronological splitting" into a concrete pandas implementation that correctly handles the multi-product panel structure of Lens 2. A date-level split that inadvertently places different product rows from the same date into different splits would create subtle cross-product leakage. Implementing the group-level date split required thinking carefully about the join structure of the panel data.

Integration

The most integrating experience was implementing the stacking ensemble, which required synthesizing knowledge from three separate domains: time-series cross-validation theory, scikit-learn pipeline architecture, and ensemble learning meta-learning theory. No single course lecture covers this combination — I had to reason from first principles, consulting the scikit-learn source code, academic papers on ensemble methods, and Pandas documentation simultaneously.

Critical Thinking

When the Stacking Ensemble's Test WAPE (44.35%) slightly exceeded Ridge's (44.38%), I initially felt the entire stacking implementation was wasted effort. But critical reflection showed the opposite: the ensemble's *Validation* WAPE was 48.06% — suggesting it overfit more to training patterns than Ridge. The fact that the ensemble happened to slightly outperform Ridge on the specific 132-day test window is a random statistical fluctuation, not a systematic performance advantage. The validation-based selection rule is robust to these fluctuations by design.

Human Dimension

Implementing the dashboard's real-time forecasting engine connected the abstract pipeline to genuine human operational utility. When I watched the Streamlit app generate a forward kitchen schedule for Monday–Sunday showing "Prepare 18 portions of LUNCH, 22 portions of BREAKFAST, 8 portions of CHICKEN CURRY...", I realized that someone in a kitchen — a real Head Chef beginning their 05:30 AM prep — might look at this screen and decide how to allocate their morning labor. The engineering quality of my feature pipeline directly affects whether that chef prepares for a comfortable lunch service or faces a stressful 1 PM stockout.

Learning How to Learn

The most valuable transferable lesson was learning how to identify and resolve a novel technical problem (the sklearn StackingRegressor incompatibility with TimeSeriesSplit) through source code reading, GitHub issue tracking, and first-principles reasoning — without being able to find an exact StackOverflow answer. This self-directed technical problem-solving process is the core skill of a professional ML engineer.

Section 4: Key Technical Skills Acquired

Skill	Demonstration
Leakage-free feature engineering	Implemented and audited shift=1 rolling pipeline; validated with correlation tests
Cyclical encoding theory	Derived sin/cos encoding from circular distance geometry principles
Custom sklearn-compatible regressor	TimeSeriesStackingRegressor with fit/predict/get_params API
Chronological panel splitting	Date-group-level split preserving product panel integrity
Meta-learner design	Ridge meta-learner on OOF predictions with TimeSeriesSplit
Model serialisation	joblib.dump / joblib.load for production artifact management

---

Section 5: What I Would Do Differently

1. **Implement Conformal Prediction intervals:** Rather than simple ±15% planning bands, I would implement conformal prediction to generate statistically calibrated confidence intervals that automatically widen for more uncertain future dates.
2. **Test more stacking base models:** Including XGBoost alongside Ridge, RF, and LightGBM in the stacking base would provide a fourth structurally diverse learner. I hypothesize the meta-learner would learn to discount XGBoost's predictions on Lens 2 (it ranked 4th) while up-weighting Ridge — potentially improving ensemble performance by 1–2% WAPE.
3. **Add automated leakage detection tests:** Rather than manually auditing the feature matrix, I would write a formal pytest suite that mathematically verifies: for every feature f and every row t, the computation of $f(t)$ only references indices $\leq t-1$ in the original series.

I certify that this Personal Learning Journey report reflects my genuine individual contribution, learning experiences, and reflections. All described technical work was performed by me in the notebooks and code assigned to my responsibility.

Student ID: IT24102664

Date: October 2026